



Desenvolvimento Web

Profª Sabrina B. Moreira



Aprendendo CSS 6

Aula 14.1: mobile first e animações

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect. The shapes are layered, with some appearing more prominent than others, and they extend towards the edges of the frame.

Mobile first

CSS: mobile first

- ❖ Mobile First é uma abordagem de desenvolvimento em que você projeta primeiro para **telas pequenas** (celulares) e depois vai expandindo para **telas maiores** (tablet, desktop).



Fonte: Vieira, 2020

CSS: mobile first

Ou seja:

- ❖ começa com o essencial (mobile)
- ❖ adiciona melhorias progressivamente com **media queries**



CSS: desktop first x mobile first

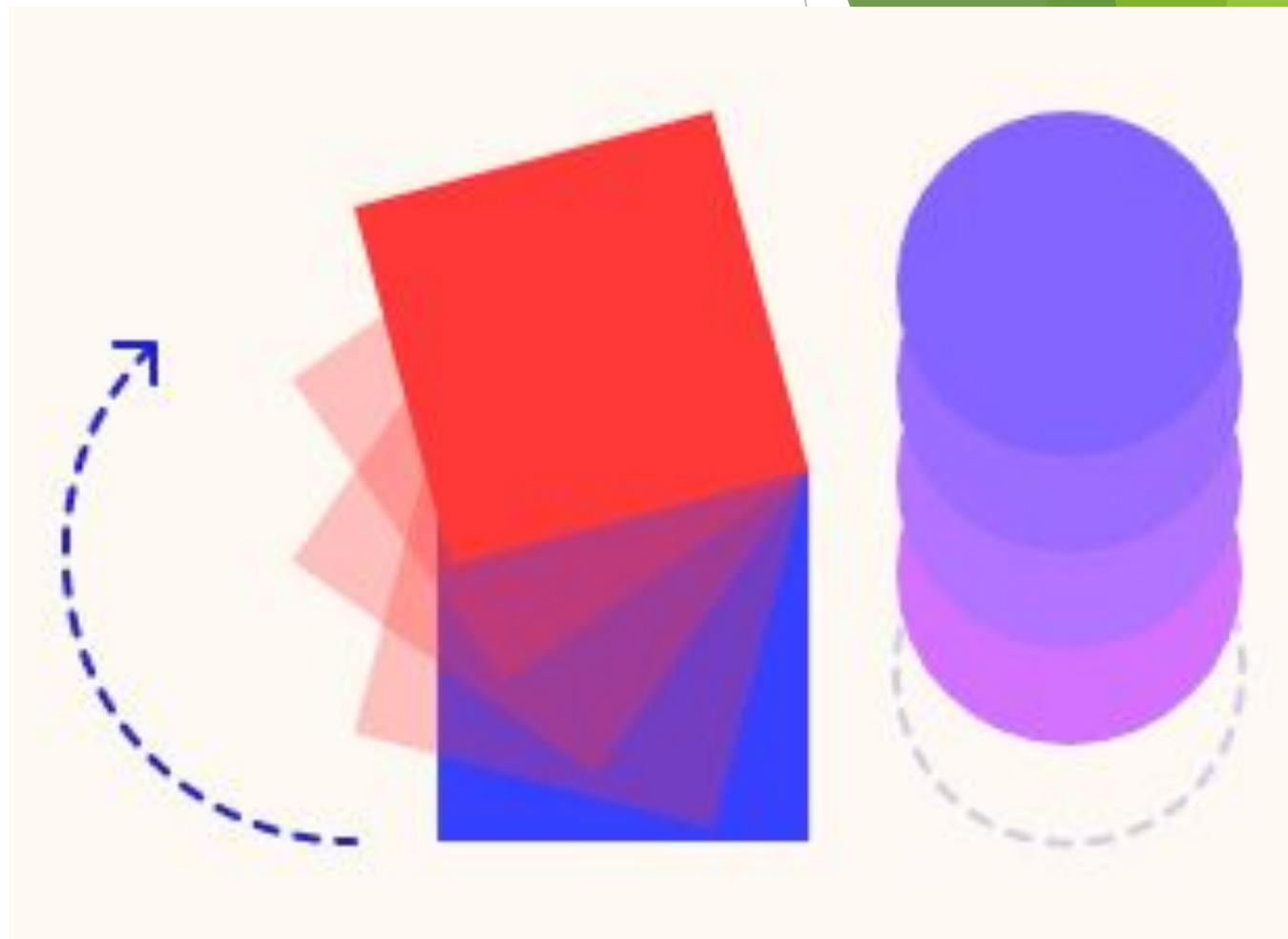
```
/* desktop first */  
.container {  
  width: 1200px;  
}  
  
@media (max-width: 768px) {  
  .container {  
    width: 100%;  
  }  
}
```

```
/* mobile first */  
.container {  
  width: 100%;  
}  
  
@media (min-width: 768px) {  
  .container {  
    width: 1200px;  
  }  
}
```

Animações

CSS: animações

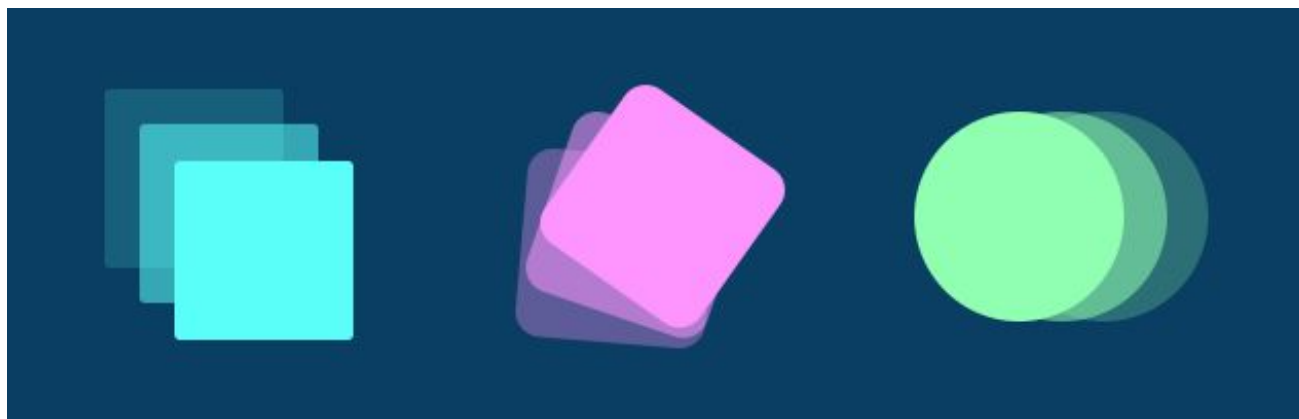
- ❖ Animações em CSS permitem alterar propriedades de um elemento ao longo do tempo, criando **movimento**, **transições suaves** e **interações visuais**, sem precisar de JavaScript.



CSS: animações

Elas são usadas para:

- ❖ Melhorar UX (feedback visual)
- ❖ Guiar atenção do usuário
- ❖ Tornar interfaces mais modernas
- ❖ Criar microinterações (hover, loading, etc.)



CSS: animações

- ❖ Nos primórdios do CSS, as ferramentas para animações eram limitadas, sendo necessário utilizar JavaScript (jQuery, setInterval, etc.), ou com **Flash** (Adobe Flash) hoje em dia obsoleto.



Fonte: canarias, 2021

CSS: animações

Animações antigas:

- ❖ Travava a página
- ❖ Difícil de manter
- ❖ Baixa performance
- ❖ Baixa fluidez

```
/* JavaScript puro */
setInterval(() => {
  element.style.left = pos + "px";
}, 16);

/* JQuery */
$("#box").animate({ left: "100px" });
```

Fonte: autora, 2026

CSS: animações

- ❖ Na década de 90, a internet era lenta, os computadores tinham **processamento limitado** e navegadores eram **inconsistentes** (Netscape x IE) e não tinha preocupação com interatividade avançada, já que as animações exigiriam muito do CPU e travariam o navegador.



CSS: animações

- ❖ O CSS3 (2010), trouxe suporte nativo a animações ao introduzir propriedades como transition, transform e o uso de @keyframes com animation, permitindo criar **efeitos visuais** ao longo do tempo com **interpolação** entre estados, com uma performance otimizada graças ao uso da aceleração de GPU.



CSS: animações

- ❖ A **aceleração por GPU** no CSS3 acontece quando o navegador delega certas animações para a **placa gráfica (GPU)** em vez de processá-las apenas pela CPU, tornando-as muito mais rápidas e suaves.
- ❖ Algumas propriedades, como transform e opacity, **não exigem recálculo** de layout nem redesenho completo da página, sendo aplicadas diretamente na etapa final de renderização chamada composição.
- ❖ Ao contrário de propriedades como top, width ou margin, que forçam a CPU a recalcular toda a estrutura da página.

CSS: animações

Transition

- ❖ Permite animar automaticamente a mudança de uma propriedade CSS entre **dois estados** (ex: normal para hover), sem precisar definir etapas intermediárias.
- ❖ Quando usar: mudanças simples como troca de cor, posição, estados...

```
button {  
  background:  blue;  
  transition: background 0.3s ease;  
}  
  
button:hover {  
  background:  red;  
}
```

Fonte: autora, 2025

CSS: animações

Transform

- ❖ Aplica transformações visuais ao elemento (mover, girar, escalar, inclinar) sem alterar o layout ao redor.

Quando usar:

- ❖ Animações performáticas
- ❖ Movimentos (em vez de usar top/left)

```
/* Move elemento */
.box {
  transform: translateX(100px);
}

/* Amplia movimento */
.box {
  transform: scale(1.2);
}

/* Gira elemento */
.box {
  transform: rotate(45deg);
}

/* Combinações */
.box {
  transform: translateX(50px) scale(1.1) rotate(10deg);
}
```


CSS: animações

Transform: translate

- ❖ Move o elemento no eixo X, Y (ou ambos), sem alterar o layout ao redor.

```
/* Move elemento no eixo x: horizontal */
.box {
  transform: translateX(100px);
}

/* Move elemento no eixo y: vertical */
.box {
  transform: translateY(100px);
}

/* Move elemento no eixo x e y */
.box {
  transform: translate(100px, 100px);
}
```

CSS: animações

Transform: scale

- ❖ Aumenta ou diminui o tamanho do elemento, no eixo X, Y (ou ambos).

```
/* Diminui o tamanho em 20% */  
.box {  
  transform: scale(0.8);  
}
```

```
/* Aumento de 50% da largura */  
.box {  
  transform: scaleX(1.5);  
}
```

```
/* Diminui 50% da altura */  
.box {  
  transform: scaleY(0.5);  
}
```

CSS: animações

Transform: rotate

- ❖ Gira o elemento em torno do seu eixo.
- ❖ Por padrão, rotate() não usa X ou Y porque ele gira o elemento no plano 2D da tela (eixo Z implícito).

```
/* gira sentido horário */  
.box {  
  transform: rotate(180deg);  
}  
  
/* gira sentido anti-horário */  
.box {  
  transform: rotate(-45deg);  
}
```

CSS: animações

Transform: rotate

- ❖ Se quisermos rotacionar 3D, precisamos indicar na definição da regra o eixo (X, Y ou Z).

```
/* inclina frente/trás */
.box {
  transform: rotateX(45deg);
}

/* efeito porta abrindo */
.box {
  transform: rotateY(45deg);
}

/* efeito similar ao rotate() */
.box {
  transform: rotateZ(45deg);
}
```

CSS: animações

Transform: rotate

- ❖ Você pode aplicar várias transformações ao mesmo tempo.
- ❖ A ordem importa para indicar a hierarquia da animação.

```
/* move, aumenta e gira */  
.box {  
  transform: translateX(50px) scale(1.1) rotate(10deg);  
}
```


CSS: animações

Transform: rotate

- ❖ Se quisermos rotacionar 3D, precisamos indicar na definição da regra o eixo (X, Y ou Z).

```
/* inclina frente/trás */  
.box {  
  transform: rotateX(45deg);  
}  
  
/* efeito porta abrindo */  
.box {  
  transform: rotateY(45deg);  
}  
  
/* efeito similar ao rotate() */  
.box {  
  transform: rotateZ(45deg);  
}
```

CSS: animações

Timing-function

- ❖ Define como a animação evolui no tempo (velocidade).
- ❖ Não muda o tempo total da animação, mas muda a distribuição da velocidade da mesma.

```
.box {  
  transition: all 0.5s;  
}
```


CSS: animações

Timing-function

- ❖ linear: constante
- ❖ ease: suave (padrão)
- ❖ ease-in: começa lento
- ❖ ease-out: termina lento
- ❖ ease-in-out: lento no início e fim

```
.box {  
  transition: all 0.5s ease-in-out;  
}
```

CSS: animações

Keyframes

- ❖ Define os estados intermediários de uma animação ao longo do tempo, como uma timeline.

Quando usar

- ❖ Animações complexas
- ❖ Múltiplas etapas

```
/* estrutura do keyframe */  
@keyframes pulse {  
  0% {  
    transform: scale(1);  
  }  
  
  50% {  
    transform: scale(1.2);  
  }  
  
  100% {  
    transform: scale(1);  
  }  
}
```

CSS: animações

Animation

- ❖ Aplica uma animação criada com *@keyframes* a um elemento e controla seu comportamento.

Principais funções

- ❖ duração
- ❖ repetição
- ❖ direção
- ❖ atraso

```
.box {  
  animation-name: pulse;  
  animation-duration: 1s;  
  animation-timing-function: ease;  
  animation-iteration-count: infinite;  
}  
  
.box {  
  animation: pulse 1s ease infinite;  
}
```

Fonte: autora, 2025

The background features abstract, overlapping green geometric shapes, primarily triangles and polygons, in various shades of green, creating a modern and dynamic visual effect.

Dúvidas?

Exercício

CSS: exercício 4

- ❖ ct-tv 1: tire o menu de botões do formulário para fora das divs de colunas (col) e adicione a classe products-form-buttons.

```
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```

```
<section class="products-form">
  <form id="tv-request-form">
    <fieldset>
      <legend>Solicitação de Modelo de TV</legend>

      <div class="col">...
    </div>

      <div class="col">...
    </div>

      <menu class="products-form-buttons">
        <button type="reset">Limpar formulário</button>
        <button type="submit">Enviar solicitação</button>
      </menu>
    </fieldset>
  </form>
</section>
```

CSS: exercício 4

- ❖ ct-tv 2: centralize os botões do formulário (text-align), utilizando a sua classe.

```
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```

```
/* ct-tvs 2 */
h1,
h2,
h3,
h4,
h5,
h6,
label,
button,
#primary-list,
.header .nav,
.products-form-buttons {
  text-align: center;
}
```


CSS: exercício 4

- ❖ ct-tv 3: alinhe os botões do formulário de tvs como centralizados na orientação de coluna (display flex e flex-direction).
- ❖ Defina largura de 80% para os botões (flex-basis).

```
/* ct-tvs 3 */  
.products-form-buttons {  
  display: flex;  
  flex-direction: column;  
  padding: 0;  
  gap: 10px;  
}  
  
.products-form-buttons button {  
  flex-basis: 80%;  
}
```

<> category_acessories.html
<> category_tvs.html
<> products.html
styles.css

CSS: exercício 4

- ❖ ct-tv 4: troque os estilos das colunas do formulário de porcentagem para display flex.
- ❖ Use a orientação coluna (flex-direction) e espaçamentos entre as linhas de 5px e colunas 10px e margin de 10px.

```
/* ct-tvs 4 */  
.col {  
  display: flex;  
  flex-direction: column;  
  gap: 5px 10px;  
  margin-bottom: 10px;  
}
```

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

CSS: exercício 4

- ❖ ct-tv 5: estruture a folha de estilos no conceito **mobile-first**.
- ❖ A direção do flex geralmente para **telas pequenas** fica melhor ordenado em colunas, e **telas grandes** em linhas.

```
/* ct-tvs 5 */
@media screen and (min-width: 768px) {
  .col, .products-form-buttons {
    flex-direction: row;
  }

  .products-form {
    max-width: 600px;
  }
}
```

```
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```

CSS: exercício 4

- ❖ all-1: padronize os espaçamentos internos (padding) em 20px, do main, cabeçalho e rodapé.
- ❖ E adicione uma cor de fundo para o rodapé (background-color).

```
/* all 1 */
main,
.header,
.footer {
  padding: 20px;
}

.footer {
  background-color: #607D8B;
}
```

```
<> category_acessories.html
<> category_tvs.html
<> products.html
# styles.css
```


CSS: exercício 4

- ❖ all-2: melhore os estilos dos inputs adicionando bordas arredondadas (border e border-radius).

```
/* all 2 */  
input {  
    border: 1px solid #3d3b3b;  
    border-radius: 5px;  
}
```

```
<> category_acessorios.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

Fonte: autora, 2026

CSS: exercício 4

- ❖ all-3: para o efeito hover de todos os links, adicione o delay de 0.8s para aplicação da opacidade.

```
/* all 3 */  
a {  
    transition: opacity 0.8s ease;  
}  
  
a:hover {  
    opacity: 0.5;  
    text-decoration: none;  
}
```

<> category_acessories.html

<> category_tvs.html

<> products.html

styles.css

CSS: exercício 4

- ❖ all-4: para o efeito hover de todas as imagens, adicione o efeito de aumentar 50% o tamanho da imagem.

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

```
/* all 4 */  
img {  
    display: block;  
    margin: auto;  
}  
  
img:hover {  
    transform: scale(1.2);  
}
```

```
h2,  
.products-list figcaption {  
    z-index: 2;  
    position: relative;  
}
```


CSS: exercício 4

- ❖ ct-tv 6: crie um keyframe de animação para rotacionar o **titulo de produtos** da página de categoria-tvs.
- ❖ Utilize o rotate x-y-x e alteração em graus.

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

```
#products-title {  
  animation: rotate 3s linear infinite;  
}  
  
@keyframes rotate {  
  0% {  
    transform: rotateX(20deg);  
  }  
  
  50% {  
    transform: rotateY(20deg);  
  }  
  
  100% {  
    transform: rotateZ(10deg);  
  }  
}
```

CSS: exercício 4

- ❖ ct-tv 7: crie um keyframe de animação para deslocar o **título de principal (h1)** da página de categoria-tvs.
- ❖ Utilize o `translate` e alteração em porcentagem.

```
<> category_acessories.html  
<> category_tvs.html  
<> products.html  
# styles.css
```

```
/* ct-tvs 7 */  
.intro h1 {  
  animation: move 3s linear infinite;  
}  
  
@keyframes move {  
  0% {  
    transform: translate(0, 0);  
  }  
  
  50% {  
    transform: translate(20%, 20%);  
  }  
  
  100% {  
    transform: translate(0, 0);  
  }  
}
```

CSS: exercício 4: resultado

Solicitação de Modelo de TV

Nome:	Ex: Maria Silva	E-mail:	Ex: maria@email.
Modelo de TV desejado:	Escolha ou digite um modelo		
Limpar formulário		Enviar solicitação	

Solicitação de Modelo de TV

Nome:	Ex: Maria Silva
E-mail:	Ex: maria@email.com
Modelo de TV desejado:	Escolha ou digite um modelo
Limpar formulário	
Enviar solicitação	

Principais dúvidas

- Como solicitar um modelo?

Principais dúvidas

- Como solicitar um modelo?

CSS: exercício 4: resultado



Fonte: autora, 2026



Salve esse código no seu email,
Google Drive ou GitHub!

Vídeo



Vídeo



Referência da aula

- ❖ W3SCHOOLS. CSS Grid Layout. W3Schools, [s.d.]. Disponível em: https://www.w3schools.com/css/css_grid.asp. Acesso em: 30 mar. 2026.

Atividade 6

- ❖ Desenvolver projeto web contento páginas HTML e CSS usando todos os conceitos que vimos até o momento, na apresentação, explicar os conceitos e mostrar como foram aplicados no código. O código deve ser apresentando funcionando em telas pequenas e grandes.
- ❖ A sala será separada em 6 equipes, cada equipe terá um relator.
- ❖ O relator deverá subir o código em seu GitHub e **enviar o link** mais os **slides** na ferramenta de trabalhos e atividades até a data da apresentação de sua equipe.
- ❖ As próximas duas aulas serão para **executar o trabalho em sala e tirar dúvidas** com a professora. Terá mais duas aulas onde todas as equipes **apresentarão**.
- ❖ Nota: Slides: 15% (grupo), Código: 15% (grupo), Apresentação: 70% (individual)

*O relator deverá entregar um papel com nome completo dos membros da sua equipe até o **final dessa aula**.*



univille.br